

🚀 مرحله اول : آشنایی با Cassandra (مباحث تئوری)

هدف این مرحله اینه که دقیقاً بدونی:

- Cassandra چیست؟
- چرا بهش نیاز داریم؟
- کاربردش چیه؟
- چه تفاوتی با دیتابیس‌های دیگه داره؟

۱. تعریف Cassandra 📺

Cassandra (کاساندر) یک پایگاه داده NoSQL است که به صورت توزیع شده طراحی شده. منظور از NoSQL پایگاه داده‌هایی هستند که:

- مدل ذخیره‌سازی منعطف‌تری نسبت به SQL دارند.
 - برای داده‌های بزرگ (Big Data) بهینه شدند.
 - توانایی مدیریت داده‌هایی با حجم بسیار بالا و متنوع رو دارند.
- کاساندر در سال ۲۰۰۸ توسط فیسبوک توسعه پیدا کرد و در سال ۲۰۰۹ به عنوان پروژه‌ی متن باز به Apache واگذار شد.

۲. ویژگی‌های کلیدی Cassandra 🎯

♦ توزیع‌شدگی (Distributed)

- کاساندر یک پایگاه داده کاملاً توزیع شده است؛ یعنی داده‌ها در چندین نود (Node) پخش می‌شن.
- هیچ نود مرکزی (Master) وجود نداره، در نتیجه نقطه شکست مرکزی نداریم. (No Single Point of Failure)

♦ مقیاس‌پذیری افقی (Horizontal Scalability)

- به راحتی می‌شه با اضافه کردن سرورهای بیشتر، ظرفیت رو افزایش داد.
- با افزایش داده‌ها و بار کاری، عملکرد همچنان بالا می‌مونه.

♦ دسترس‌پذیری بالا (High Availability)

- داده‌ها در چند نود به صورت همزمان ذخیره می‌شن. (Replication)
- در صورت خرابی یک یا چند نود، داده‌ها در دسترس می‌مونن.

♦ مدل داده‌ای منعطف (Flexible Data Model)

- مدل داده‌ای Cassandra به صورت «جدول» و بر اساس «Column-Family» هست.
- نیازی به تعریف دقیق Schema (ساختار جدول) از ابتدا نیست و Schema به راحتی قابل تغییر هست.

✿ ۳. تفاوت Cassandra با دیتابیس‌های رابطه‌ای (SQL)

ویژگی	Cassandra (NoSQL)	دیتابیس‌های رابطه‌ای (مثل MySQL)
مدل داده	Column-Family	Relational Tables (جدول‌های رابطه‌ای)
مقیاس‌پذیری	افقی (Horizontal Scaling)	بیشتر عمودی (افزایش منابع سرور)
تراکنش‌ها	محدود (Consistency کم)	قوی (Consistency بالا)
Join و کوئری‌های پیچیده	پشتیبانی محدود	پشتیبانی کامل و قدرتمند
مناسب برای	حجم بالای داده، Real-time و توزیع‌شده	داده رابطه‌ای، کوئری‌های پیچیده و تحلیلی

• ۴. نمونه کاربردهای Cassandra در دنیای واقعی

سازمان	کاربرد
Netflix	مدیریت داده‌های کاربران، ویدیوها و وضعیت تماشا
Instagram	ذخیره‌سازی کامنت‌ها، لایک‌ها و ارتباطات
Apple	ذخیره داده‌های iCloud و سرویس‌های مرتبط
Spotify	مدیریت Playlist و رفتار کاربران

- علت استفاده شرکت‌ها از Cassandra
 - پاسخگویی سریع به تعداد بالای درخواست‌ها
 - دسترس‌پذیری بالا و تضمین تداوم سرویس
 - مدیریت حجم عظیمی از داده‌ها بدون افت سرعت

📌 ۵. دلایل انتخاب Cassandra در سیستم‌های توزیع‌شده

در درس «سیستم‌های توزیع‌شده» هدف این‌ه که سیستمی بسازیم که:

- در برابر خرابی (Fault-Tolerant) مقاوم باشه.
- در زمان افزایش بار کاری، به راحتی مقیاس‌پذیر باشه.

- داده‌ها رو در چند نقطه جغرافیایی ذخیره کنه و در دسترس باشه.
- Cassandra دقیقاً برای رفع این نیازها طراحی شده:
- داده‌ها به‌طور همزمان در چندین نود ذخیره می‌شن.
- قابلیت تحمل خرابی (Fault Tolerance) و بازیابی خودکار داده‌ها رو داره.
- در صورت نیاز به افزایش ظرفیت، فقط نودهای جدید به شبکه اضافه میشن.

مرحله دوم: معماری Cassandra (تئوری)

در این مرحله معماری Cassandra و نحوه کارکرد آن در سیستم‌های توزیع‌شده رو یاد می‌گیریم.

مواردی که باید به‌خوبی بدونی و در ارائه خودت توضیح بدی این‌ها هستن:

۱. معماری کلی Cassandra (Ring)

۲. پروتکل Gossip

۳. پارتیشن‌بندی داده‌ها (Partitioner)

۴. تکرار داده‌ها (Replication)

۵. استراتژی‌های Replication

۱. معماری Ring در Cassandra

معماری Cassandra بر اساس ساختار حلقه‌ای (Ring) بنا شده:

- داده‌ها به‌طور مساوی در بین نودهای مختلف توزیع میشن.
- هر نود مسئولیت بخشی از داده‌ها رو داره.
- در Cassandra نود اصلی (Master) وجود نداره و همه نودها برابر هستن (Peer-to-peer).
- اضافه یا حذف کردن نود بسیار راحت انجام می‌شه و داده‌ها به صورت اتوماتیک دوباره توزیع میشن.

نمونه یک معماری Ring :

Node ۱ → Node ۲ → Node ۳

↑ ↓

Node ۶ ← Node ۵ ← Node ۴

هر نود داده‌هایی با توکن مشخص رو مدیریت می‌کنه.

هر داده بر اساس کلید (Partition Key) توکن می‌گیره و در نود مربوطه قرار می‌گیره.

۲. پروتکل Gossip

Cassandra از پروتکلی به نام **Gossip Protocol** برای ارتباط بین نودها استفاده می‌کنه:

- پروتکل Gossip روشی هست که نودها وضعیت خودشون و نودهای دیگر (مثل سالم بودن یا نبودن) رو به‌طور پیوسته به هم اطلاع میدن.
- هر نود به‌طور دوره‌ای اطلاعاتش رو به چند نود دیگه می‌فرسته.
- این اطلاعات به سرعت در کل شبکه پخش میشه و همه نودها از وضعیت یکدیگر اطلاع دارن.

این مکانیسم کمک می‌کنه تا:

- Cassandra خیلی سریع از خرابی یک نود مطلع بشه.
- در صورت خرابی، داده‌ها از نودهای دیگر خونده بشن.
- به سرعت داده‌ها در نودهای جدید توزیع بشن.

۳. پارتیشن‌بندی داده‌ها (Partitioning)

پارتیشن‌بندی در Cassandra توسط **Partitioner** انجام می‌شه:

- زمانی که داده‌ای رو در Cassandra ذخیره می‌کنیم، هر داده یک کلید (**Partition Key**) داره.
- Cassandra این کلید رو به یک توکن هاش (Hash Token) تبدیل می‌کنه و بر اساس این توکن، مشخص می‌کنه که داده باید در کدام نود ذخیره بشه.
- بنابراین وقتی دنبال داده‌ای می‌گردیم، Cassandra به سرعت می‌دونه داده دقیقاً کجاست و مستقیماً به سراغ همون نود می‌ره.

مثال ساده از Partition Key و توکن:

نود مسئول	توکن هاش (Token)	کلید داده (Key)
Node ۱	۱۲۷۵۸۲۳۱۲۳	User۱۲۳
Node ۲	۱۴۵۸۹۲۱۵۸۸	User۴۵۶
Node ۳	۱۷۳۵۶۱۸۲۶۳	User۷۸۹

- با این روش، داده‌ها به‌طور یکنواخت در کل شبکه توزیع می‌شن.

۴. تکرار داده‌ها (Replication)

Cassandra داده‌ها رو به صورت همزمان در چند نود ذخیره می‌کنه تا در صورت خرابی یک نود، داده از دست نره و همیشه در دسترس باشه:

- به این ویژگی Replication می‌گیم.
- پارامتر مهمی به نام Replication Factor (عامل تکرار داده‌ها) داریم که تعداد نسخه‌های تکراری از یک داده رو مشخص می‌کنه.

مثلاً:

- اگر Replication Factor = ۳ باشه، هر داده در ۳ نود متفاوت ذخیره می‌شه.
- اگر یک نود از دسترس خارج شد، هنوز ۲ نسخه دیگه در نودهای دیگه در دسترس هست.

۵. استراتژی‌های Replication در Cassandra

Cassandra دو استراتژی رایج برای Replication داره:

● SimpleStrategy

- برای کلاسترهای تک Data Center استفاده می‌شه.
- داده‌ها به سادگی و به ترتیب توکن در نودهای بعدی ذخیره می‌شن.

مثال:

```
CREATE KEYSPACE my_keyspace WITH replication = {  
  'class': 'SimpleStrategy',
```

```
'replication_factor': '3'
};
```

● NetworkTopologyStrategy

- برای شبکه‌هایی با چندین Data Center استفاده می‌شود.
- می‌تونی تعیین کنی هر Data Center چند کپی از داده داشته باشه.

مثال برای دو Data Center (DC1 و DC2):

```
CREATE KEYSPACE my_keyspace WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'DC1': 3,
  'DC2': 2
};
```

📄 خلاصه کلیدی این مرحله برای ارائه:

«معماری Cassandra یک معماری حلقه‌ای (Ring) و بدون نود مرکزی (Masterless) است. نودها با استفاده از Gossip Protocol اطلاعات را سریعاً به اشتراک می‌گذارند. داده‌ها بر اساس Partition Key بین نودها توزیع می‌شوند و برای اطمینان از دسترس‌پذیری بالا، چند نسخه از داده‌ها (Replication) در نودهای مختلف نگهداری می‌شود.»

📁 مرحله سوم: مدل داده‌ای Cassandra (تئوری)

در این مرحله با ساختار داده‌ها در Cassandra آشنا می‌شیم. مدل داده‌ای Cassandra شباهت‌هایی به SQL دارد ولی تفاوت‌هایی هم دارد که خیلی مهمه بهش توجه کنی.

در این بخش باید موارد زیر رو خوب متوجه بشی و بتونی توضیح بدی:

۱. Keyspace (فضای کلیدها)

۲. Table (جدول)

۳. Row (سطر) و Column (ستون)

۴. Primary Key (کلید اصلی)

○ Partition Key

○ Clustering Column

۱. Keyspace

یک **Keyspace** در Cassandra مثل یک **Database** در SQL هست.

- هر Keyspace یک فضای مجزا برای ذخیره‌ی مجموعه‌ای از جداول هست.
- هنگام ساخت Keyspace ، **Replication Strategy** رو هم تعیین می‌کنیم.

مثال ایجاد یک Keyspace ساده:

```
CREATE KEYSPACE my_keyspace
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': '3'
};
```

در این مثال:

- **my_keyspace** نام فضای داده‌ای است.
- داده‌ها در ۳ نود به طور همزمان ذخیره میشن. (Replication Factor=۳)

۲. Table (جدول)

یک **Table** در Cassandra شبیه جدول‌های دیتابیس‌های SQL هست.

- Table از تعدادی **ستون (Column)** و **سطر (Row)** تشکیل شده.
- در Cassandra نیازی نیست از ابتدا ساختار دقیقی برای جدول تعریف کنیم؛ می‌تونیم بعداً ستون‌ها رو اضافه کنیم.

مثال ایجاد یک جدول ساده (برای ذخیره کاربران):

```
USE my_keyspace;

CREATE TABLE users (
    user_id int PRIMARY KEY,
    first_name text,
    last_name text,
    email text
```

);

- ستون‌های جدول user_id, first_name, last_name, email :
- ستون user_id به عنوان **Primary Key** انتخاب شده است.

۳. Row و Column 📄

- **Row (سطر)**: نشان‌دهنده یک رکورد (مثلاً یک کاربر خاص) است.
- **Column (ستون)**: یک ویژگی از رکورد را نشان می‌دهد.

برای مثال:

email	last_name	first_name	user_id
ali@example.com	Rezaei	Ali	۱
sara@example.com	Ahmadi	Sara	۲

- هر ردیف (Row) یک کاربر هست.
- هر ستون (Column) یک ویژگی از اون کاربر رو نگهداری می‌کنه.

۴. Primary Key (کلید اصلی) 🔑

Primary Key در Cassandra شامل دو قسمت مهمه:

- **Partition Key (کلید پارتیشن‌بندی)**
- **Clustering Column (ستون دسته‌بندی)**

● Partition Key

- مشخص می‌کنه که داده‌ها در کدام نود ذخیره شوند.
- انتخاب مناسب Partition Key باعث توزیع مناسب داده‌ها و افزایش کارایی سیستم می‌شه.

● Clustering Column

- داده‌ها رو درون هر پارتیشن مرتب می‌کنه.
- اجازه می‌ده تا داده‌ها رو با سرعت بیشتری بازیابی کنیم.

✅ مثال با Partition Key و Clustering Column :


```
CREATE TABLE user_posts (
  user_id int,
  post_id int,
  content text,
  created_at timestamp,
  PRIMARY KEY (user_id, created_at)
) WITH CLUSTERING ORDER BY (created_at DESC);
```

در این مثال:

- **user_id** نقش Partition Key رو داره.
 - همه پست‌های یک کاربر خاص، در یک پارتیشن (یک نود) ذخیره می‌شه.
 - **created_at** به عنوان Clustering Column استفاده می‌شه.
 - پست‌ها رو به صورت نزولی (جدیدترین اول) مرتب می‌کنه.
- بنابراین کوئری زیر بسیار سریع انجام می‌شه:

```
SELECT * FROM user_posts WHERE user_id = 10 ORDER BY created_at DESC;
```

چون داده‌های کاربر ۱۰ در یک پارتیشن ذخیره و مرتب شده.

❧ ۵. تفاوت کلیدی با دیتابیس‌های رابطه‌ای (RDBMS)

ویژگی	Cassandra	SQL
کلید	Partition Key و Clustering Column	فقط Primary Key
اسکیمای جداول	منعطف و قابل تغییر	ثابت و محدود
Join بین جداول	به شدت محدود	راحت و قابل اجرا

به همین دلیل در Cassandra معمولاً داده‌هایی که به هم مرتبط هستن رو در یک جدول ذخیره می‌کنیم (Denormalization).

در این اسلاید مقایسه‌ای بین **Cassandra** به عنوان یک پایگاه داده توزیع شده‌ی NoSQL و پایگاه داده‌های رابطه‌ای سنتی مثل MySQL یا PostgreSQL انجام می‌دیم.

- اولین تفاوت اصلی در ساختار داده‌ست.
RDBMS بر پایه جدول‌های ساختاریافته و وابستگی به اسکیمای سفت و سخت کار می‌کند.
ولی Cassandra یک مدل ستون‌محور و نیمه‌ساخت‌یافته دارد که سطرها می‌توانند ستون‌های متفاوتی داشته باشند.
- از نظر مقیاس‌پذیری، RDBMS معمولاً به صورت عمودی (Vertical) مقیاس‌پذیر می‌شود، یعنی با ارتقاء سخت‌افزار.
در حالی که Cassandra کاملاً افقی (Horizontal) مقیاس‌پذیر؛ می‌توانیم با اضافه کردن نودهای جدید، ظرفیت رو بالا ببریم.
- در RDBMS یک نود مرکزی داریم که اگر از کار بیفتد، کل سیستم دچار اختلال می‌شود.
اما در Cassandra همه نودها با هم برابرند (peer-to-peer) و هیچ نقطه شکست مرکزی (No SPOF) نداریم.
- و در نهایت، در RDBMS برای تضمین صحت داده‌ها از ویژگی‌های ACID استفاده می‌شود،
ولی Cassandra با مدل AP از CAP Theorem کار می‌کند و در عوض قابلیت دسترس‌پذیری بالا (Availability) رو تضمین می‌کند.

♦ CAP Theorem چیه؟

یه اصل مهم در پایگاه داده‌های توزیع‌شده که می‌گه:

یک سیستم توزیع‌شده نمی‌تونه هم‌زمان این سه ویژگی رو با هم داشته باشه:

۱. Consistency (یکسان بودن داده‌ها در همه نودها)
۲. Availability (همیشه پاسخ دادن، حتی موقع خرابی نودها)
۳. Partition Tolerance (تحمل شکست شبکه، مثلاً قطعی بین نودها)

 Cassandra کدوم دوتا رو انتخاب می‌کند؟

Availability + Partition Tolerance ♦ → مدل AP

یعنی چی؟

- حتی اگر ارتباط بعضی از نودها قطع بشه (partition)، Cassandra همچنان جواب می‌ده (availability)
- اما ممکنه داده‌ها در لحظه یکسان نباشن (Consistency کم‌رنگ‌تر)

بخش عملی

استارت مجدد کانتینر

```
docker start cassandra-node
```

ورود به محیط CQLSH داخل کانتینر

```
docker exec -it cassandra-node cqlsh
```

ایجاد جدول

```
CREATE TABLE users (  
    user_id int PRIMARY KEY,  
    first_name text,  
    last_name text,  
    email text  
);
```

درج (Insert)

```
INSERT INTO users (user_id, first_name, last_name, email)  
VALUES (1, tannaz, 'farahani', tf@example.com');
```

خواندن (Select)

```
-- نمایش همه کاربران:  
SELECT * FROM users;  
  
-- فیلتر بر اساس user_id:  
SELECT * FROM users WHERE user_id = 1;
```

بروزرسانی (Update)

```
UPDATE users  
SET email = 'ali.rezaei@iut.ac.ir'  
WHERE user_id = 1;
```

حذف (Delete)

```
DELETE FROM users WHERE user_id = 1;
```